

# Statistical Modeling of Disfluent Speech

BTech Project I Endterm Presentation

Rohit Khatri Akshay Malwey  
Dr. Mrinmoy Bhattacharjee

Department of Computer Science & Engineering

April 27, 2026

# Outline

- 1 Introduction
- 2 Literature Survey
- 3 Proposed Work
- 4 Results and Analysis
- 5 Conclusions and Future Work
- 6 References

# What is Disfluent Speech?

- Repetitions, prolongations, blocks, and interjections (e.g., “um”, “uh”).
- Over **70 million people worldwide** stutter; many more experience temporary disfluencies.
- Example: “I want to ...to go to the the store.”

# ASR Systems Fail on Disfluencies

- Whisper, Siri, Alexa achieve  $\geq 95\%$  accuracy on fluent speech.
- On disfluent speech, **error rates double or triple**.
- Disfluencies are often “normalized away” – repetitions deleted, pauses ignored.

# Why This Project?

- Very few annotated disfluent datasets exist.
- Current detection methods are inflexible or need clean transcripts.
- Goal: Generate synthetic data, analyse ASR failures, and build robust detection.

# Approach 1: Pattern Matching + Statistical Filter

## Technical contribution:

- Detects repairs using only lexical clues (word matches, fragments, editing terms like “um”).
- Builds repair patterns on-the-fly; uses augmented POS tagger to filter false positives.

## Limitations:

- Requires hand-annotated word fragments; no prosody.
- Does not handle fresh starts (abandoned utterances).

# Approach 2: Cleanup Language Model

## Technical contribution:

- Extends N-gram language model to treat disfluencies as hidden events.
- Conditions word predictions on a “cleaned” (edited) context.

## Limitations:

- Minimal impact on overall word error rate.
- Filled pauses often predict next word better than cleaned context.

# Approach 3: BLSTM + Integer Linear Programming

## Technical contribution:

- Bidirectional LSTM with a 17-state tag space for repetitions and corrections.
- Uses Integer Linear Programming (ILP) to enforce structural constraints.

## Limitations:

- Relies on pre-defined resources (POS tags, word embeddings).
- Limited adaptability to new domains.



# Approach 4: Unconstrained Dysfluency Modeling

## Technical contribution:

- Hierarchical transcription and detection without perfect ground truth.
- Uses Unconstrained Forced Aligner (UFA) + DTW-based Text Refresher.

## Limitations:

- Computationally expensive.
- Performance degrades for high-disfluency, low-resource languages.

# Approach 5: LLM + Prompt Engineering

## Technical contribution:

- Uses GPT-4o, Gemini, etc. with zero-shot, few-shot, chain-of-thought prompts.
- No task-specific training.

## Limitations:

- High computational cost; not real-time.
- Performance varies with disfluency density and language.

# Proposed Approach & Work Done so far

- **Synthetic data generation:** Rule-based text augmentation (sound/syllable repetition, blocks, interjections).
- **Evaluation:** N-gram language models ( $n=3,4,5$ ) to measure perplexity.
- **TTS conversion:** Generate audio from synthetic text; compare with real TORGO.
- **Future detection:** A Hybrid detection approach.

## Synthetic Disfluent Text Generation:

- Two custom Python functions:
  - `sound_repetition`: Repeats first letter (e.g., “s-s-sun”).
  - `syllable_repetition`: Repeats first two letters (e.g., “su-su-sun”).
- Also insert blocks (“...”) and interjections (“um”, “uh”).
- Input: fluent sentences from TORGO.
- Libraries: `random`, `re`, `nltk`, `datasets`, `librosa`, `soundfile`.

## N-gram Language Model Evaluation:

- Train trigram, 4-gram, 5-gram on synthetic vs. real TORGO transcripts.
- Use MLE and Laplace smoothing (via `nltk.lm`).
- Compute **perplexity** – lower means more predictable / natural.
- Results are shown in the next section.

# Proposed Approach & Work Done so far Contd.

- Synthetic text → audio using Azure TTS or Coqui TTS.
- Compare with real TORGO audio using:
  - Whisper transcription (WER).
  - Listening tests for naturalness.
- Initial observation: Repetitions are preserved, but TTS sounds more artificial than real speech.

# Perplexity Comparison (Real vs. Synthetic)

| Model         | Real TORGO | Synthetic |
|---------------|------------|-----------|
| Trigram (n=3) | 245        | 5120      |
| 4-gram (n=4)  | 198        | 4980      |
| 5-gram (n=5)  | 175        | 4850      |

**Table:** Perplexity (lower is better). Synthetic data has perplexity 5000 – much worse than real.

# Analysis of Synthetic Text Quality

- **Frequency:** We can match overall disfluency rate by tuning probabilities.
- **Position:** Real disfluencies cluster at word boundaries; ours are uniformly random.
- **N-gram overlap (BLEU):** Preliminary BLEU score = 0.32 (low similarity).
- **Why?** N-gram LMs are local; even with real data perplexity  $\approx 100$ , but synthetic is an order of magnitude worse due to simple random rules.

# TTS Output – Initial Observations

- Whisper transcription of generated audio:
  - Repetitions: often transcribed correctly (e.g., “the the”).
  - Prolongations: sometimes split (e.g., “ssssun” → “s sun”).
- Real TORGO audio has natural pitch/timing variations; TTS lacks this.
- Next step: waveform manipulation (WSOLA) on fluent TORGO audio to add disfluencies directly.



# Conclusions

- Simple rule-based synthetic text generation is feasible but produces data statistically very different from real disfluent speech (high perplexity, low BLEU).
- N-gram LMs are too weak to model disfluent patterns effectively.
- TTS from synthetic text yields intelligible but unnaturally prosodic audio.

# Future Work

- Understand working of state-of-the-art TTS models (e.g., Whisper) – how they handle disfluencies and where they make errors.
- Study disfluent speech patterns (repetitions, prolongations, blocks) for better synthetic data generation – how different disfluencies behave in real speech.
- Create a hybrid model that detects each disfluency type with better accuracy.

# References I

- Heeman, P. A., & Allen, J. F. (1994). Detecting and correcting speech repairs. *ACL*.
- Stolcke, A., & Shriberg, E. (1996). Statistical language modeling for speech disfluencies. *ICASSP*.
- Zayats, V., Ostendorf, M., & Hajishirzi, H. (2016). Disfluency detection using a bidirectional LSTM. *arXiv:1604.03209*.
- Lian, J., et al. (2023). Unconstrained dysfluency modeling. *ASRU*.
- de Lima, P. L. S., & Campelo, C. E. C. (2024). Disfluency detection and removal via LLMs. *STIL*.
- Mujtaba, D., et al. (2024). Inclusive ASR for disfluent speech. *arXiv:2406.10177*.
- Radford, A., et al. (2023). Robust speech recognition via weak supervision. *ICML*.

# Thank you

**Rohit Khatri & Akshay Malwey**